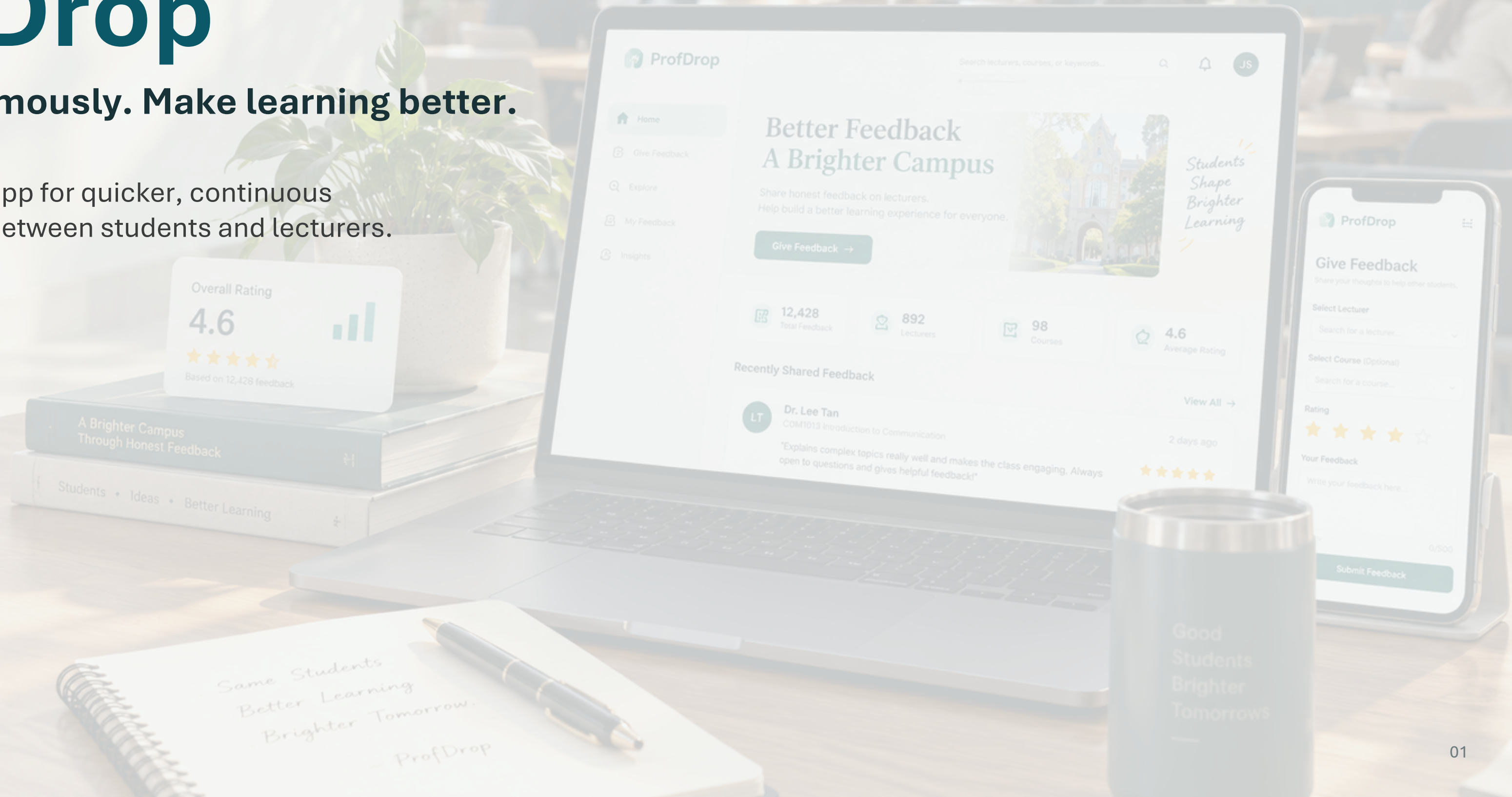


Anonymous lecturer feedback

ProfDrop

Say it anonymously. Make learning better.

A full-stack web app for quicker, continuous communication between students and lecturers.



Presentation roadmap

1

Problem

Why normal SETs are not enough on their own

2

Solution

How ProfDrop improves feedback flow

3

Application

Student and lecturer user flows

4

Build

Frontend, backend, API, storage and security

5

Delivery

Testing, Docker, CI/CD, deployment and cost

6

Demo

What we will show live after the slides

The problem: SET feedback is useful, but too delayed



Current SET process

- Only available at certain times
- Usually has a closing deadline
- Feedback goes to admin first
- Lecturers receive a summary later

What students need

After class

Feedback should be possible while the experience is still fresh.

Safe voice

Some students are nervous to speak in class.

Direct route

Lecturers need to see useful comments quickly.

Continuous improv

Small changes can happen before the end of term.

Non-functional requirements

01

Privacy and security

Do not store student identity with feedback. Protect lecturer access with authentication, Function keys and course ownership checks.

02

Data persistence

Keep lecturer, course and feedback records in Azure Table Storage. Store lecturer profile images in Azure Blob Storage.

03

Usability

Keep student feedback quick to submit and make lecturer courses and feedback clear to review.

Configuration

Storage connection strings and Function keys stay outside committed source code.

Our solution: continuous anonymous feedback



Student

Select course, choose category,
write feedback anonymously



Lecturer

Log in, view assigned courses, filter
feedback by category

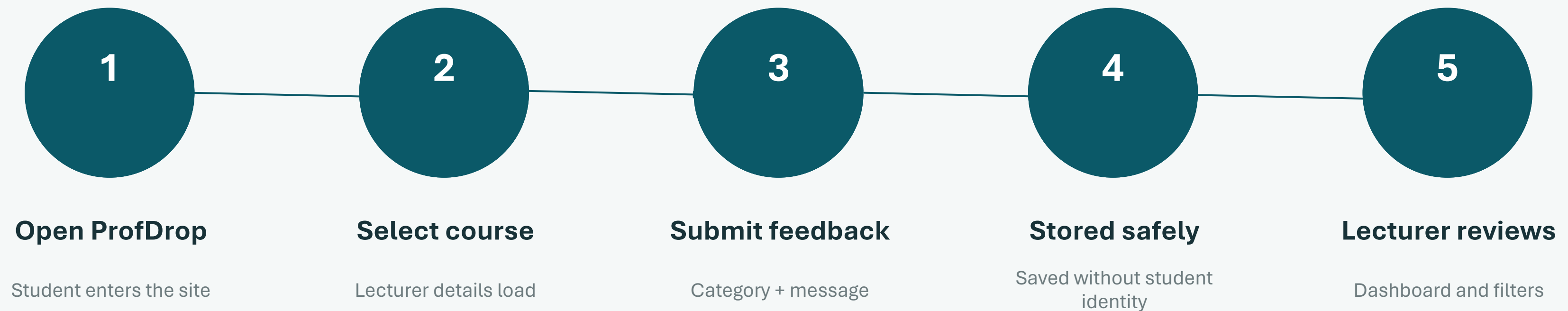


Impact

Feedback can be reviewed after
every class, not only at semester
end

Design principle **Keep it simple enough for students to use quickly, but structured enough for lecturers to act on.**

Application journey: from student comment to lecturer action



Key difference from normal SETs: feedback does not wait for a survey window or an admin summary before reaching the lecturer.

Actual screen flow: student side

ProfDrop

HomeGive FeedbackDashboard

Lecturer Login

Anonymous Lecturer Feedback

ProfDrop

Say it anonymously. Make learning better.

Share constructive feedback with your lecturers in a simple and anonymous way.

Give Feedback

Lecturer Login



For Students

Share anonymous feedback about your lecturer and course experience.



For Lecturers

View student feedback for your courses and filter it by category.

© 2026 - ProfDrop

Home page

Two clear starting points: Give Feedback and Lecturer Login.

ProfDrop

HomeGive FeedbackDashboard

Lecturer Login

Give Feedback

Share anonymous feedback about your lecturer.

Course

CLDV6212 - Cloud Development



Lecturer

Ms H. Mthembu

h.mthembu@college.ac.za

Feedback Category

Explanation

Your Feedback

The practical examples after the theory helped me understand the Azure Functions work much better.

Submit Feedback

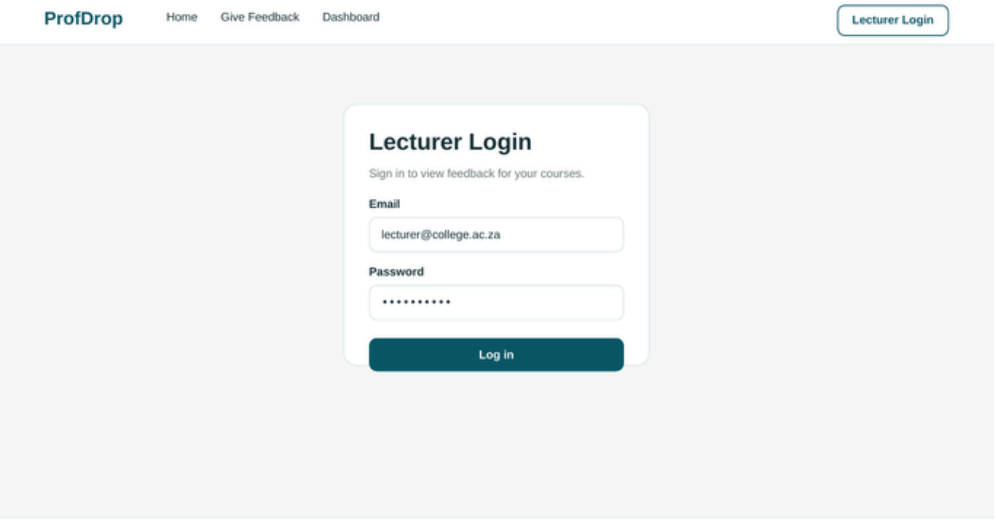
© 2026 - ProfDrop

Student feedback form

Course, lecturer preview, category and anonymous message.

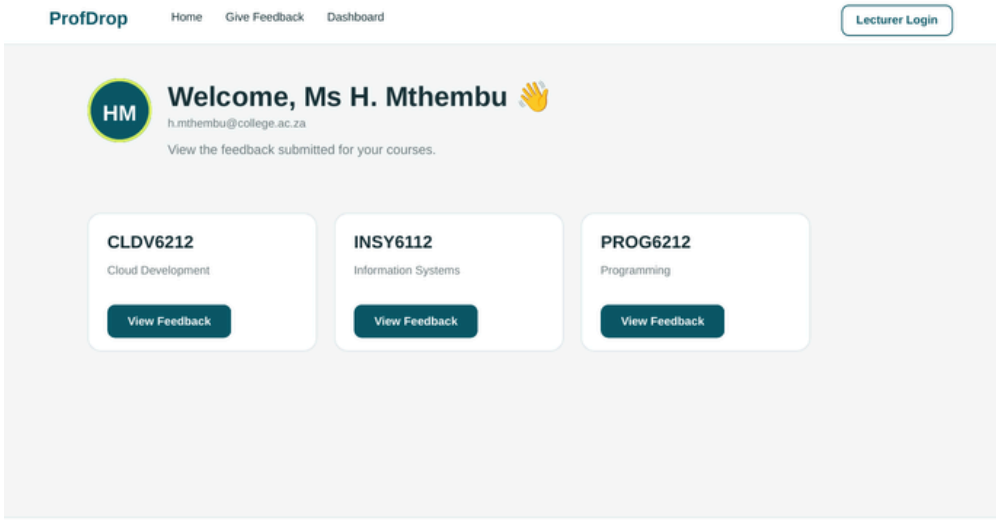
06

Actual screen flow: lecturer side



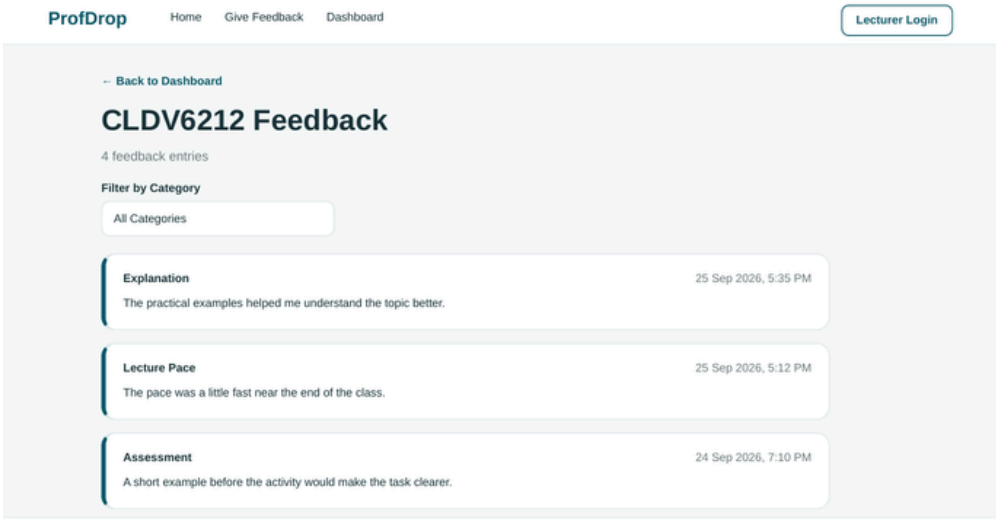
Lecturer Login

Protected dashboard access using lecturer credentials.



Dashboard

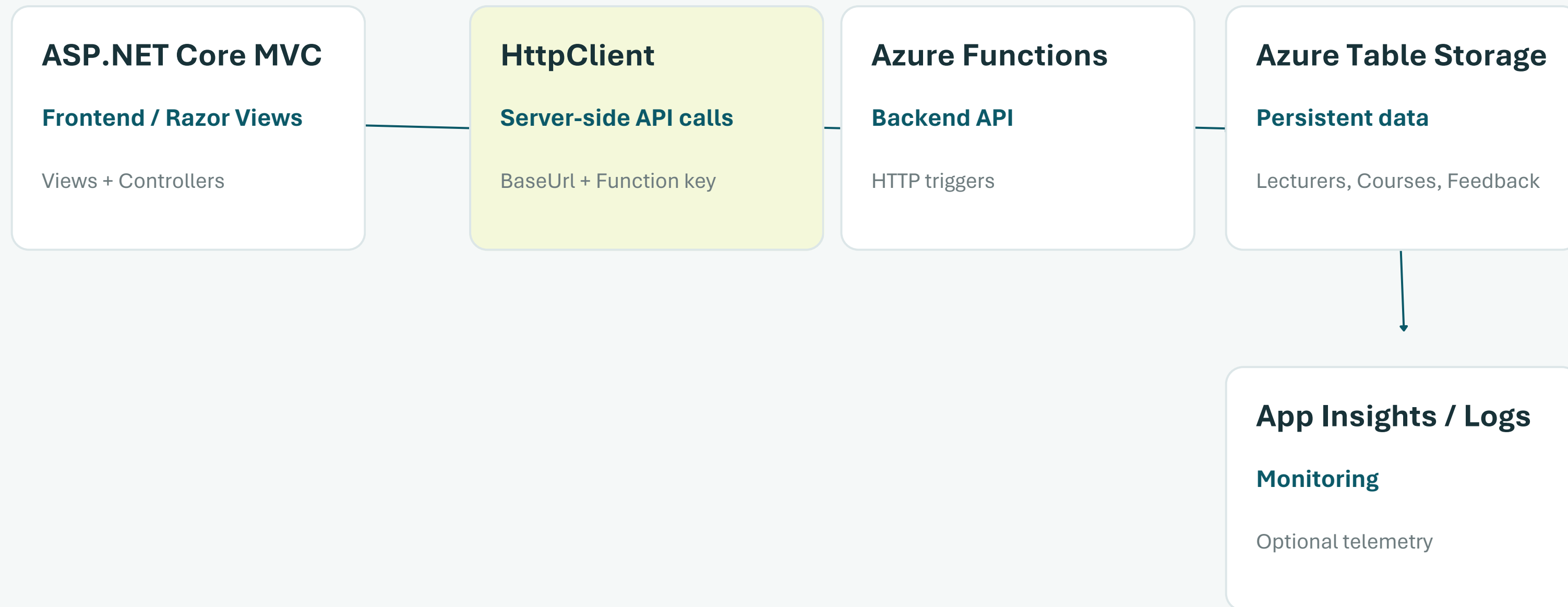
Shows the logged-in lecturer and courses assigned to them.



Course Feedback

Lecturer can view feedback entries and filter by category.

Technical architecture



Local development: Visual Studio + Functions runtime + Azurite for storage emulation

Final deployment: separate containerized services with environment variables

Backend API: endpoints that support the user flows

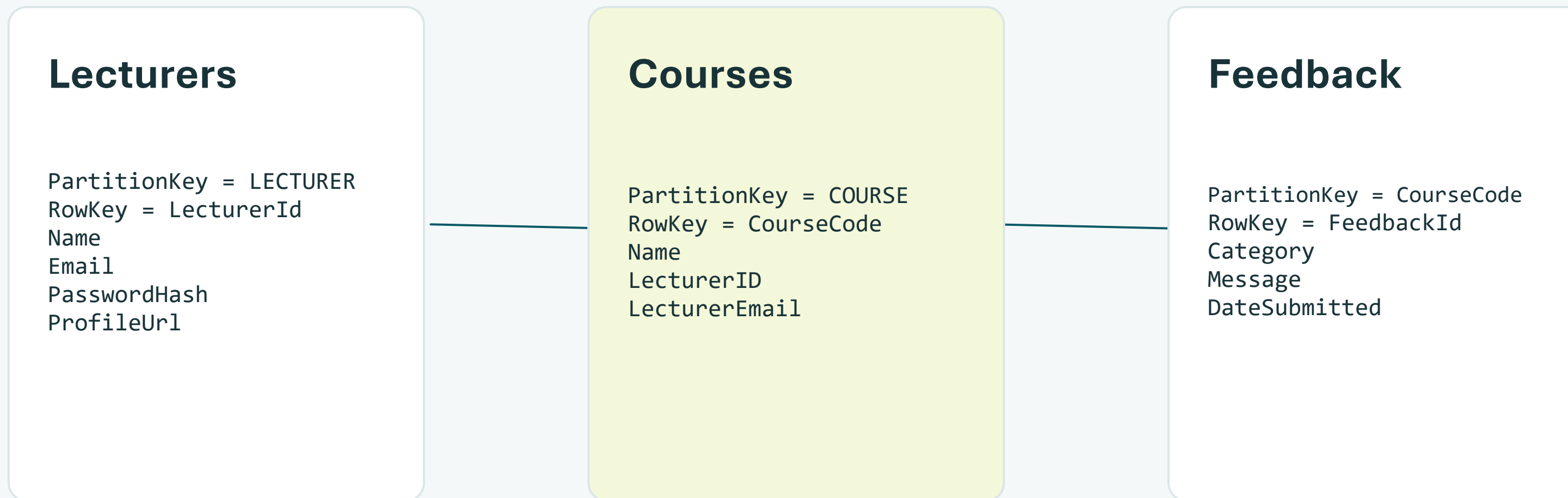
Endpoint map

GET	/courses	Student course dropdown
GET	/courses/{courseCode}	Lecturer preview on course select
POST	/feedback	Anonymous feedback submission
POST	/lecturers/login	Lecturer authentication
GET	/lecturers/{id}/courses	Dashboard courses
GET	/lecturers/{id}/courses/{code}/feedback	Feedback list with access check
POST	/lecturers, /courses	Admin setup data

Why routes matter

- Students only need anonymous public routes.
- Lecturer data uses protected routes.
- Course ownership is checked before feedback is returned.
- Routes are separated by responsibility: courses, feedback, lecturers.

Data model and persistence



Storage choice Table Storage fits the project because the data is simple, key-based and cheap/free-tier friendly for a small feedback app.

Code spotlight: anonymous feedback validation

```
if (data == null ||
    string.IsNullOrEmpty(data.CourseCode) ||
    string.IsNullOrEmpty(data.Category) ||
    string.IsNullOrEmpty(data.Message))
{
    return BadRequest("Course code, category and message are
required.");
}

var feedback = new Feedback
{
    PartitionKey = courseCode,
    RowKey = Guid.NewGuid().ToString(),
    Category = category,
    Message = message,
    DateSubmitted = DateTimeOffset.UtcNow
};

await feedbackTableClient.AddEntityAsync(feedback);
```

1. Required fields

The API rejects empty course, category or message values before saving.

2. Anonymous by design

The saved feedback object contains no student name, email or ID.

3. Persistent storage

Each response gets a unique RowKey and is saved in the Feedback table.

Security, privacy and access control



Anonymous feedback

No student identity fields are stored with feedback records.



Lecturer cookie auth

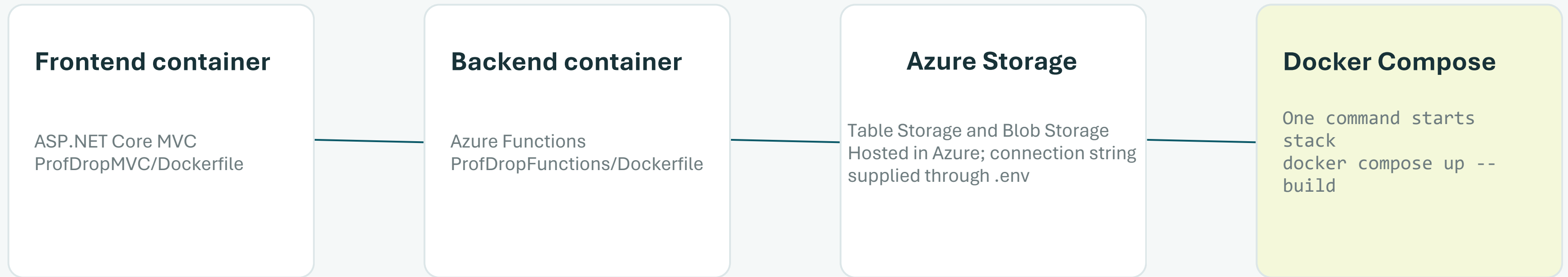
MVC uses cookie authentication for logged-in lecturers.



Ownership check

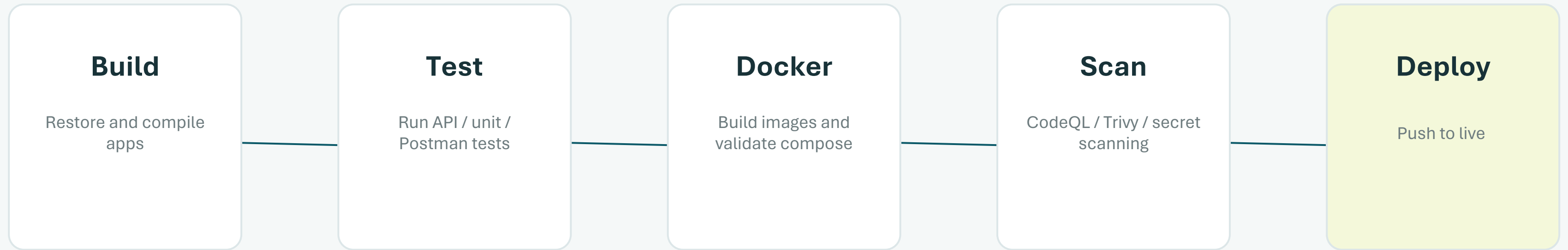
A lecturer can only view feedback for courses linked to their LecturerID.

Containerization and local development



docker compose up --build Expected outcome: frontend, backend API and local storage start together on a Docker network.

CI/CD pipeline and deployment approach



Collaboration and Git workflow

Switch branches/tags

Find or create a branch...

Branches

Tags

✓ main

st10467156/hameeda

st10496167/khadija

default

View all branches

Complete student feedback MVC

Khadija10496-2 committed last week

Align course and feedback APIs with MVC

Khadija10496-2 committed last week

Implement course and feedback functions

Khadija10496-2 committed last week

added .env.example

hameedahamid committed 20 minutes ago · ✓ 5 / 5

Merge pull request #7 from hameedahamid/st10467156/hameeda

hameedahamid authored 39 minutes ago · ✓ 4 / 4

Completed docker compose

hameedahamid committed 44 minutes ago · ✓ 5 / 5

40 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Merge pull request #8 from hameedahamid/st10467156/hame... ProfDrop CI #21: Commit 33b52c0 pushed by hameedahamid	main		17 minutes ago 1m 59s	...	
✓	Merge pull request #8 from hameedahamid/st10467156/hame... CodeQL Advanced #19: Commit 33b52c0 pushed by hameedahamid	main		17 minutes ago 2m 2s	...	
✓	added .env.example CodeQL Advanced #18: Pull request #8 opened by hameedahamid	st10467156/hameeda		20 minutes ago 1m 52s	...	
✓	added .env.example ProfDrop CI #20: Pull request #8 opened by hameedahamid	st10467156/hameeda		20 minutes ago 2m 18s	...	
✓	Merge pull request #7 from hameedahamid/st10467156/hame... ProfDrop CI #19: Commit fceb230 pushed by hameedahamid	main		41 minutes ago 1m 16s	...	
✓	Merge pull request #7 from hameedahamid/st10467156/hame... CodeQL Advanced #17: Commit fceb230 pushed by hameedahamid	main		41 minutes ago 2m 6s	...	
✓	St10467156/hameeda CodeQL Advanced #16: Pull request #7 opened by hameedahamid	st10467156/hameeda		45 minutes ago 1m 45s	...	
✓	St10467156/hameeda ProfDrop CI #18: Pull request #7 opened by hameedahamid	st10467156/hameeda		45 minutes ago 1m 21s	...	
✓	Merge pull request #6 from hameedahamid/st10467156/hame... CodeQL Advanced #15: Commit b9837e3 pushed by hameedahamid	main		Today at 7:50 PM 2m 0s	...	

Future improvements after the MVP

Notifications

Alert lecturers when new feedback arrives.

Class-level prompts

Allow feedback after a specific lecture/session.

Trends dashboard

Show categories over time and repeated issues.

Helpful summaries

Group similar feedback without losing exact comments.

Reply / action status

Lecturers can mark feedback as reviewed or actioned.

Institution reports

Admin can view anonymous high-level trends only.

Main future goal: move feedback from “once-off survey” to “ongoing learning improvement”.

Docker Compose challenges

Function keys

CHALLENGE

The API kept Function-level authorization, so MVC requests needed a valid key.

WHAT FIXED IT

Mounted the Functions secrets folder and sent the key in the x-functions-key header.

Azure Storage settings

CHALLENGE

The containers needed Azure settings without putting real credentials in Git.

WHAT FIXED IT

Loaded the connection string from .env. Azure Storage stays in Azure; no emulator container is used.

```
docker compose up --build
```

Starts the MVC and Functions containers, which connect to the Azure Storage account.